AgileUI

AI Agent 驱动的 **APP 动态 UI 编辑平台**

让每一次微小需求，都可以敏捷落地



AI 搞定 0 → 1 的创造，我们解决 1 → 100 的持续迭代

JSON 驱动 | 可视化 Canvas | AI Agent

一个按钮，流程要跑一整周

当客户只是想把首页按钮往上挪一点，这在 APP 研发里，是一条‘设计 – 开发 – 发版’的瀑布流程。

客户以为

1 分钟

拖一下，或说一句话

实际要花

1 周 +

6 个环节 · 3 个角色 · 多端发版

每天都在发生的循环

- **客户 / 甲方** 「把按钮移到上面，更醒目一点」
- **产品 / 设计** 写 PRD、改 Figma、对齐还原度
- **开发 / 测试** 改原生代码、编译、提审、装包、验证

每周重复几十次 · 全部塞进发版窗口

为什么一直没被自动化？

AI 与低代码都在卷「从 0 到 1 生成页面」；上线后 1 → 100 的持续迭代，成了一条没人修的瀑布链。

每一次流转，都在丢失信息

传统 Scrum 里，一个 UI 需求要被翻译 4 次，才能到达用户手机端验收。

口述需求 → PRD → Figma → 原生代码 → 编译发版 → 真机验收

01

信息折损

客户 / BA / 设计 / 开发
四个角色，需求永远对不上

02

改 1 秒 vs 改 1 周

Figma 拖一下几秒钟
原生 UI 要按周发版、测试

03

AI 写得快，改不动

一句「按钮大一点」可能整页重写
无 Schema 约束，接不进现有工程

本质

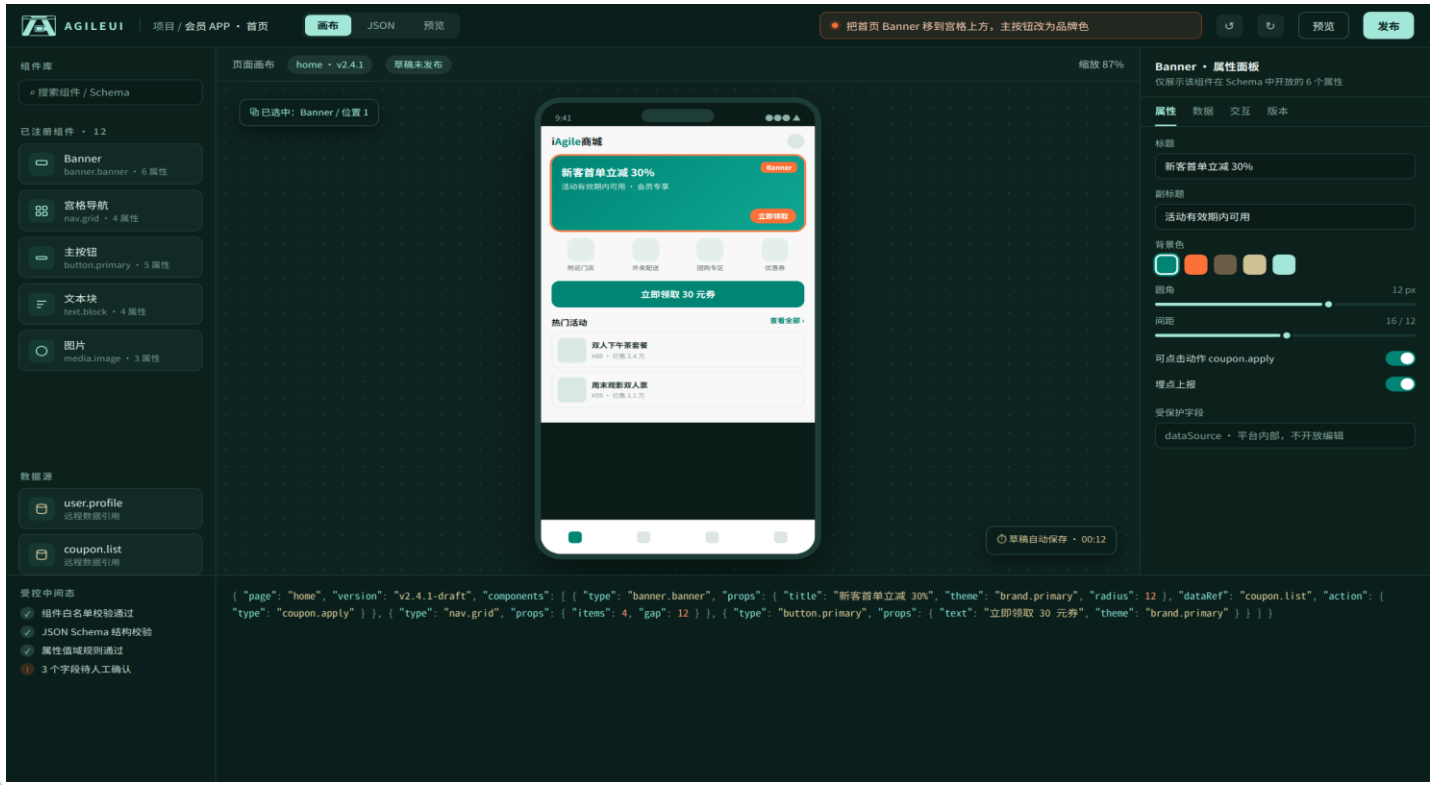
静态代码把 UI 写死了 —— 界面改动与业务代码死死绑定，一次微调就要牵动整套工程。

AgileUI：把 UI 变成可配置的数据

研发封装组件，业务拖拽或说话，线上 APP 秒级生效 —— 不改一行源码。

AgileUI Studio · Web 可视化编辑台

LIVE UI



界面说明

01 组件库 / 白名单

研发预先注册可用组件与开放属性

02 可视化画布

拖拽编排，画布即时预览，不写代码

03 属性面板

只暴露 Schema 允许的字段，越界即拦截

04 AI 指令 + JSON

一句话生成局部 Patch，配置即源码

安全沙箱：不触碰底层业务逻辑

JSON 驱动 + 可视化 Canvas + AI Agent

先代码（模块化组件），后设计（可视化配置）—— 让页面成为一份可审阅的数据。

01

JSON 驱动

受控中间态 · SSOT

组件类型 / 属性 / 动作 / 数据引用

统一定义，Schema 校验后发布

→

02

可视化 Canvas

所见即所得的操作入口

拖拽编排 → 属性配置 → 即时预览

人与 AI 写入同一份页面配置

→

03

AI Agent

另一种操作入口

自然语言 → 目标定位 → 局部 Patch

不重写整页，不生成任意代码

依据

组件注册表 + JSON Schema + 数据引用规则，把「界面」从「代码」里解耦：

渲染端只消费已发布的配置，由本地已注册组件负责呈现。

AI 不写生产代码，只改一份配置

凭什么敢让 AI 碰线上界面？四道闸门，人工最后确认。

01
目标定位
组件类型 + 页面结构
锁定要改的路径

→

02
生成 Patch
RFC 6902 JSON Patch
只描述增量变化

→

03
规则校验
白名单 / 路径 / 属性
值域必须合法

→

04
Schema 校验
结构 + 业务规则
非法即拒绝并返回反馈

→

05
人工确认
草稿预览 → 接受 / 撤销
发布后才成为生效版本

AI 可以做

✓ 定位已注册组件与开放属性

✓ 生成局部补丁，写入当前草稿

✓ 与拖拽共用同一套预览 / 发布链路

AI 不可以做

× 生成或执行任意 JavaScript / 原生代码

× 凭空创建未注册组件、开放未注册动作

× 跳过校验、人工确认与发布控制

从一周，到一分钟

同一份配置，多端一致渲染；0 编译、0 发版、0 源码改动。

1 分钟

小幅 UI 调整交付周期 · 原需 1 周

0

次重新编译 / 打包 / 发版

1

份配置 · 业务与工程共同语言

BEFORE

改设计 › 改代码 › 编译 › 提审 › 装包 › 验证

AFTER

拖拽 / 说一句话 › Schema 校验 › 草稿预览 › 确认发布

六款工具，各管 SDLC 一段

AgileUI 不是又一个「AI 生成 UI」—— 它补的是上线之后的持续迭代。

SDLC 阶段	Figma Make	Cursor Design Mode	Builder.io	M3E · Canvas	AGenUI	AgileUI
需求表达	●	—	●	●	—	●
UI / UX 设计	●	—	●	—	—	●
代码实现	—	●	●	—	●	●
发版测试	—	—	●	—	—	●
客户端渲染	—	—	—	—	●	●
上线运维	—	—	—	—	●	●

只有 AgileUI 贯穿「需求 → 设计 → 代码 → 发版 → 渲染 → 上线后迭代」全链路，且以受控配置交付。

我们和它们的区别

别人交付「原型 / 代码」，AgileUI 交付「受控、可校验、可发布的配置变更」。

产品	核心定位	输出物	主要用户
Figma Make	用 AI 把想法变成交互原型设计稿	可交互原型设计稿	设计师
Cursor Design Mode	在运行页面指定修改，AI 改代码	UI 代码	开发者
Builder.io	设计系统 + 可视化编辑 + 内容发布	Web 页面 / UI 代码	开发 / 设计 / 运营
M3E · Canvas	组件画布，辅助组织 AI 输入	更明确的 Prompt	PM / 运营
AGenUI	生成式 UI 跨平台原生渲染 SDK	iOS / Android 原生 UI	开发者
AgileUI	**把频繁变动的想法变成受控、可迭代的配置变更**	**校验后的 JSON 配置**	**客户 / 产品**

差异化

我们不替代设计师与开发者，而是成为需求意图与代码实现之间「可审阅的中间态」——
目标用户第一次扩展到客户、业务与运营。

产品路线图

2026.10 — 2027.08 · 每一阶段都有可验证的交付物。



MVP 闭环

移动端演示闭环
打通发布与回滚链路
补齐异常提示与自测

一致性与可追溯

种子色推导全局主题
JSON 变更转自然语言日志
变更可审计、可回溯

打通设计上游

Figma → JSON 插件
组件映射与版本兼容
降低企业迁移成本

团队试点与规模协作

权限审批 / 灰度发布
客户试点验证付费意愿
评估效率提升与使用频率

已完成 组件清单与注册表 · 协议与 JSON Schema · Web 可视化编辑画布 · 预览渲染

进行中：移动端演示闭环 · AI Agent 修改链路 · 客户验证

团队介绍

架构 / 全栈 / Agent / 设计与 Demo —— 小团队，完整闭环。

黄

黄 卓

项目总负责人 · 架构设计

PMP® · PMI-ACP® · 5 年+ 技术项目管理

负责AgileUI从产品定位、技术架构、MVP原型开发、AI Agent搭建的整体设计

杨

杨庆锐

Web 编辑器与移动端开发

实现 JSON 预览与画布双向联动

C 端预览独立页及手机端适配

完成移动端 Demo

张

张雅萌

场景 Demo 与主设计

中国香港教育大学 AIET-Master 在读

熟练搭建 Agent，独立实现 0 → 1 MVP 闭环

负责 PPT 与视频主设计、痛点挖掘

林

林培辉

团队成员

职责与代表项目待补齐

提交决赛材料前替换此卡

项目制协作：产品定位 → 架构与协议 → 前后端实现 → 场景 Demo → 商业化路径

已经跑通，正在找真实客户

MVP 核心链路可演示；下一步是把效率提升，变成可复用的商业价值。

已完成 01

- 组件清单与组件注册表
- 协议与 JSON Schema
- Web 可视化编辑画布
- 预览渲染

进行中 02

- 移动端演示闭环
- AI Agent 修改链路
- 客户验证

下一步 03

- 垂直行业组件模板
- Figma 一键同步插件
- 灰度发布与权限

目标客户

饱受需求变更折磨的 APP 产研团队与甲方 —— 客户 / 业务 / 运营直接掌控线上 UI

试点验证指标：效率提升幅度 · 使用频率 · 付费意愿

让每一次微小需求，
都可以敏捷落地



AI 搞定 0 → 1 的创造，AgileUI 解决 1 → 100 的持续迭代。

我们在找

- **种子客户试点**
- **工程与合作资源**
- **投融资对话**